Reporte Técnico de Actividades Práctico-Experimentales Nro. 4

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Melissa Contento, Jaime Landázuri, Alejandro Padilla Espinoza, Doménica Rojas, Ivett Zaragocín
Asignatura	Sistemas Digitales
Ciclo	5
Unidad	1 – Microcontroladores para sistemas embebidos
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	4
Tipo	Grupal
Título de la Práctica	Uso de interrupciones en un sistema de alerta doméstica
Nombre del Docente	Ing. Luis Darío Sinche C.
Fecha	2026-11-06
Horario	Viernes 11h30 a 13h30
Lugar	Laboratorio de computación (simulación en Wokwi y prueba en la placa ESP32 del grupo)
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Construir un sensor de puerta con un pulsador y un LED que detecte la apertura mediante una interrupción externa del ESP32, y comprobar experimentalmente por qué la interrupción no pierde eventos que el sondeo sí pierde y por qué hace falta el antirrebote.

Objetivos específicos:

- Configurar una interrupción externa con attachInterrupt() y el flanco FALLING.
- Escribir una ISR corta, con IRAM_ATTR, variables volatile y patrón bandera.
- Explicar con las mediciones propias la diferencia entre sondeo e interrupción y el efecto del rebote.

3. Materiales, Reactivos, Equipos y Herramientas

- Simulación: Wokwi en el navegador y wokwi-cli, con el ESP32 DevKit V1, un pulsador, un LED rojo y una resistencia de 330 Ω.
- Placa real: ESP32 DevKit V1 de 30 pines con cable USB de datos, protoboard, pulsador de 4 patas, LED rojo de 5 mm, resistencia de 330 Ω y cables jumper.
- Software: Arduino IDE con el paquete esp32 de Espressif 3.x, sin librerías externas, y GitHub para la entrega.

4. Procedimiento / Metodología Ejecutada

El pulsador hace de sensor de puerta (presionado = puerta abierta), el LED es la luz de alarma y el Monitor Serie es la notificación. Primero se cargó el diagram.json del Anexo A de la guía en Wokwi y se probó el Código 1 (sondeo). Los conteos de las Tablas A y B salen de wokwi-cli; las capturas, de corridas manuales en el navegador; luego el Código 2 (interrupción) con T_REBOTE = 0 y con T_REBOTE = 50; la Parte C repite el Código 2 en la placa real del grupo.

4.1. Esquema del circuito

El circuito es el del Anexo A de la guía: el pulsador va entre D21 y GND usando patas en diagonal, con la resistencia pull-up interna, y el LED se conecta a D19 a través de una resistencia de 330 Ω, con el cátodo a GND. Ningún pin elegido es de arranque, de memoria flash o solo de entrada [1].

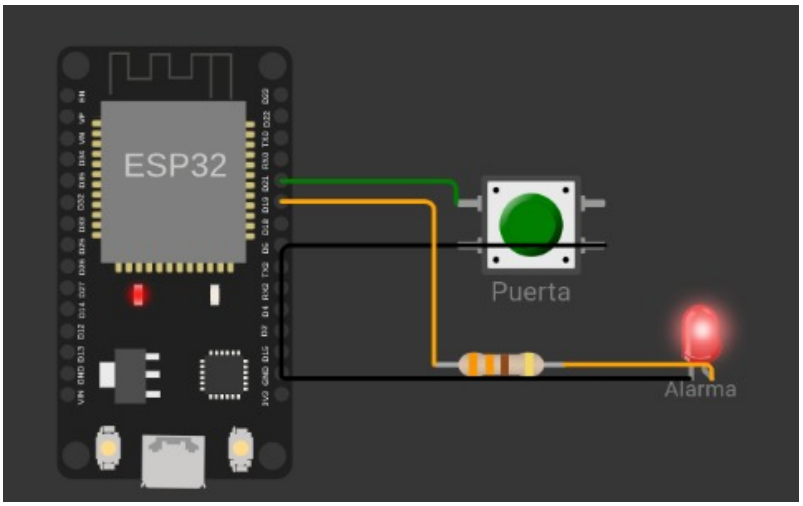


Figura 1: Diagrama del circuito en Wokwi: ESP32 DevKit V1, pulsador en D21 y LED con resistencia de 330 Ω en D19

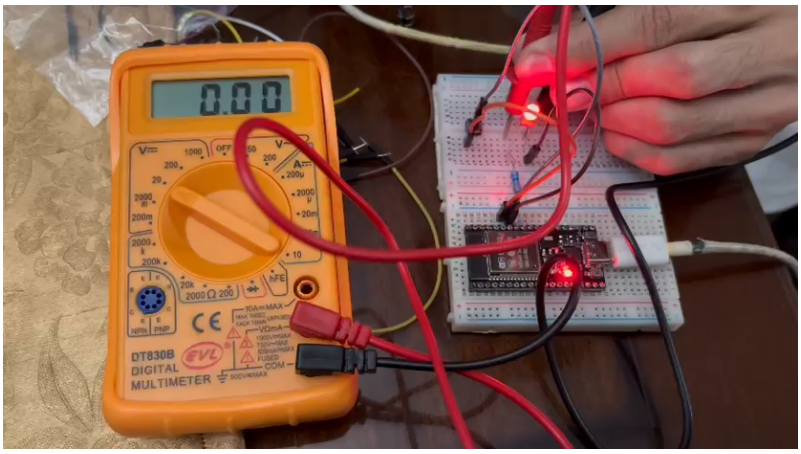


Figura 2: Montaje en la placa ESP32 real del grupo, con el LED de alarma encendido y el multímetro sobre el LED

Enlaces del trabajo:

- Proyecto en Wokwi del Código 1 (sondeo): wokwi.com/projects/479994010623459329
- Proyecto en Wokwi del Código 2 (interrupción): wokwi.com/projects/479992357138333697
- Repositorio de GitHub, carpeta AP4 con los dos códigos, el diagram.json, las capturas del Monitor Serie y la foto del montaje: github.com/AlejandroTatum/sistemas-digitales-unl (carpeta AP4)

5. Resultados

Cada prueba consistió en 10 toques cortos y rápidos del pulsador, en unos 10 segundos. En Wokwi se repitió cada prueba dos veces con ritmos distintos: en el intento 1, cada toque duró 150 ms con 850 ms de pausa; en el intento 2, 120 ms con 930 ms de pausa. El pulsador de Wokwi simula el rebote mecánico [2].

5.1. Parte A. Sondeo

Tabla A. Sondeo: aperturas reportadas con 10 pulsaciones.

Intento	Pulsaciones realizadas	Aperturas reportadas
1	10	0
2	10	1

De las 20 aperturas reales de los dos intentos, el sondeo detectó una sola.

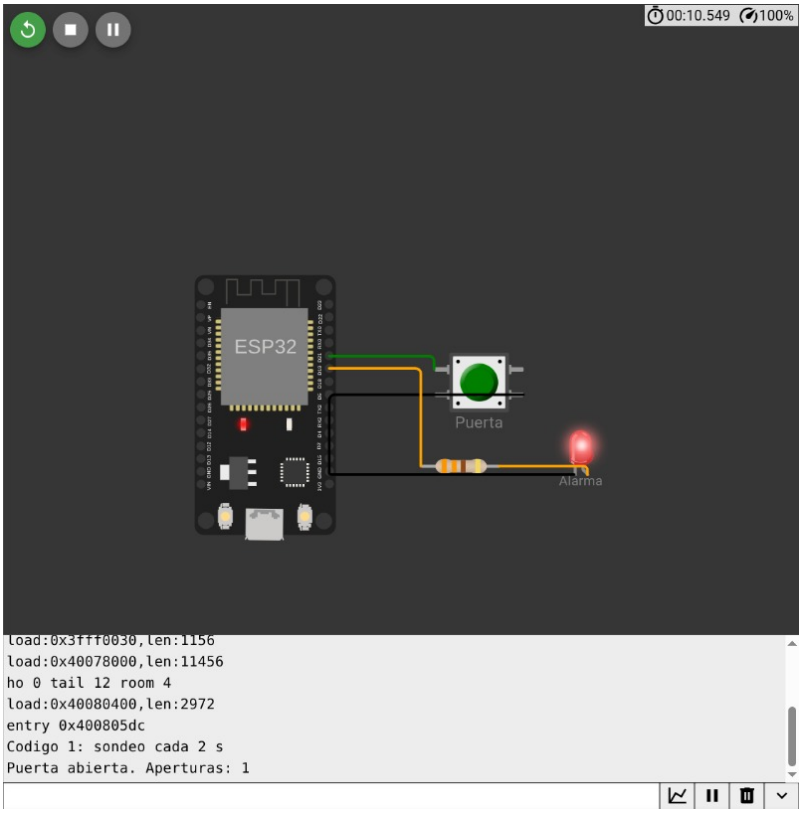


Figura 3: Monitor Serie del Código 1 (sondeo) tras 10 pulsaciones

5.2. Parte B. Interrupción y antirrebote en Wokwi

Tabla B. Interrupción en Wokwi: eventos contados con 10 pulsaciones.

T_REBOTE (ms)	Eventos, intento 1	Eventos, intento 2	¿Más, menos o igual a 10?
0	905	963	Más de 10
50	20	20	Más de 10

Con T_REBOTE = 0, cada rebote dispara la ISR y el contador llega a entre 90 y 96 eventos por toque; una corrida aparte en el navegador terminó en 876 eventos, del mismo orden. Con T_REBOTE = 50 el conteo baja de unos 930 a 20 eventos por intento: 2 por toque, y el contador Eventos sube con cada toque, así que no se pierde ninguna apertura. La captura del navegador con 10 clics también terminó en 20 eventos.

El segundo evento de cada toque aparece al soltar: el contacto también rebota al abrirse, y esos rebotes producen flancos de bajada que llegan más de 50 ms después del flanco aceptado al presionar, porque cada toque duró entre 120 y 150 ms. El filtro por tiempo descarta los rebotes de un mismo cambio, pero no distingue el rebote de soltar de una apertura nueva.

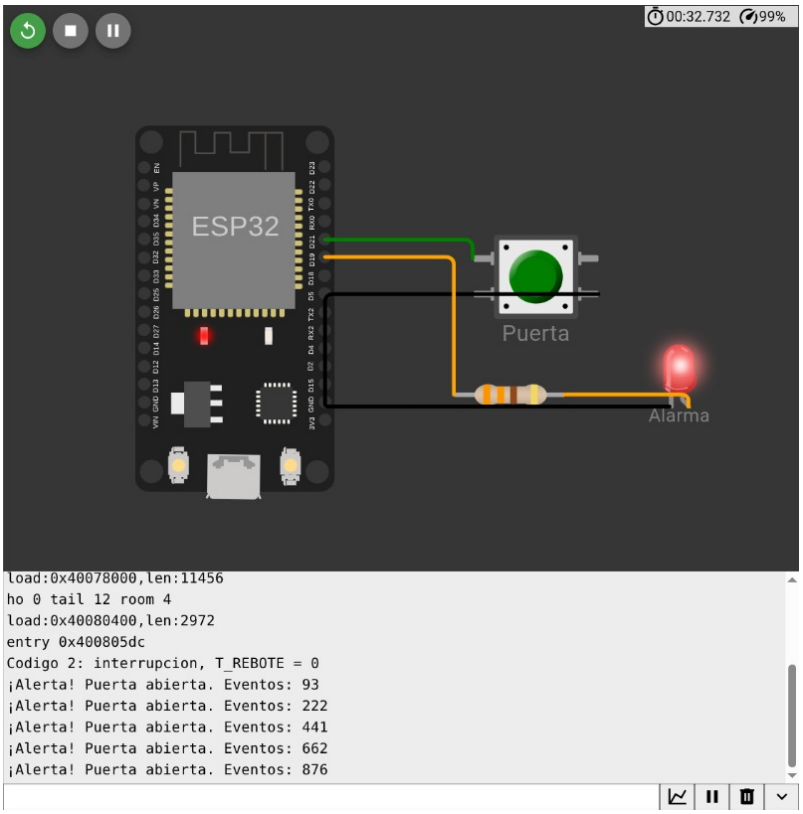


Figura 4: Monitor Serie del Código 2 con T_REBOTE = 0

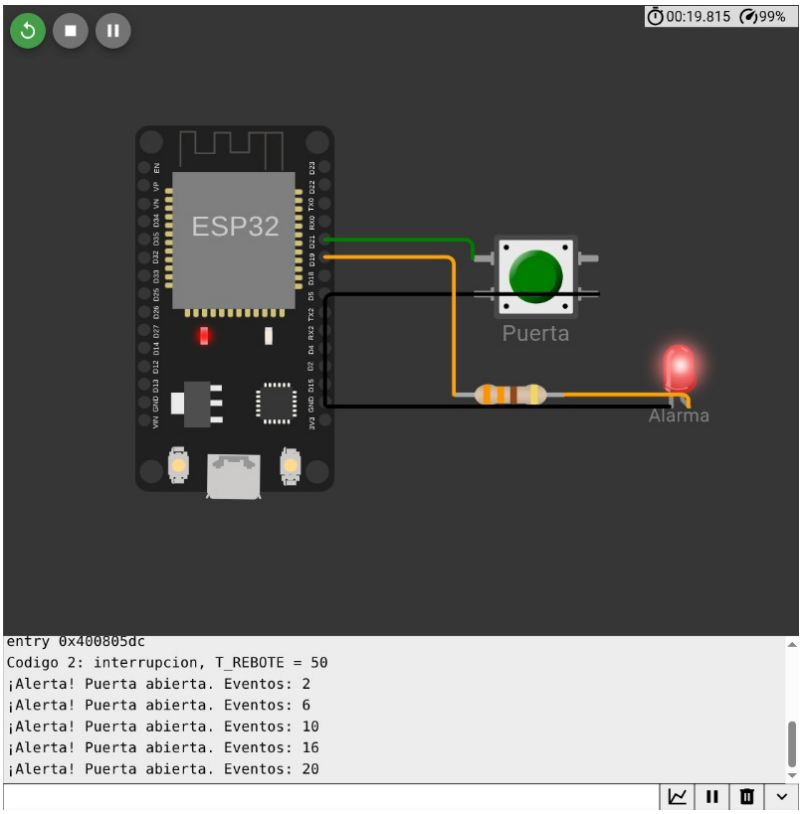


Figura 5: Monitor Serie del Código 2 con T_REBOTE = 50

5.3. Parte C. Placa ESP32 real

El grupo no tuvo Monitor Serie con la placa, así que la prueba se hizo mirando el LED de alarma y midiendo su voltaje con un multímetro. Se grabó un video de cada condición con 10 pulsaciones, y los cambios de estado del LED se contaron cuadro a cuadro.

Tabla C. Placa real: comportamiento del LED con 10 pulsaciones.

T_REBOTE (ms)	Intento 1	Intento 2	Observaciones
0	Eventos no medidos; el LED cambió de estado 5 veces	No realizado (un solo video por valor)	Cambios a los 2, 4, 6, 8 y 12 s del video; 1,91 a 1,92 V en el LED encendido y 0,00 V apagado
50	Eventos no medidos; el LED cambió de estado 2 veces	No realizado (un solo video por valor)	El LED se encendió a los 2,8 s y se apagó a los 6,8 s

Valor esperado por teoría, no medido: con T_REBOTE = 0 el contador debería pasar de 10, porque un pulsador mecánico rebota varios milisegundos y cada flanco dispara la ISR [3]; con T_REBOTE = 50 debería quedar entre 10 y 20, según si el contacto también rebota al soltar, como ocurrió en Wokwi.

El montaje funcionó: el LED responde al pulsador mientras el loop() está en el delay(2000). Sin embargo, el LED no sirve como contador, porque el loop() lo cambia a lo sumo una vez cada 2 s aunque la ISR haya contado decenas de eventos; por eso los cambios quedan separados por múltiplos de 2 s y la placa no permite comparar los conteos de 0 y 50 ms como la Tabla B.

6. Preguntas de Control

6.1. Con el Código 1, ¿por qué se perdieron aperturas?

Porque el loop() pasa casi todo el tiempo dentro del delay(2000) y solo lee el pin un instante cada 2 s. Si el toque empieza y termina entre dos lecturas, el programa nunca ve el pin en 0; por eso la Tabla A muestra 0 y 1 aperturas de 10.

6.2. ¿Por qué el Código 2 no pierde aperturas aunque también tiene delay(2000)? ¿Quién vigila el pin?

Lo vigila el periférico GPIO del ESP32 por hardware: al detectar el flanco de bajada avisa a la CPU, que pausa el programa, ejecuta la ISR y vuelve al mismo punto, aunque el loop() esté en el delay [4]. La ISR solo cuenta y levanta la bandera, sin que el programa tenga que consultar el pin [5]; el trabajo lento queda para el loop().

6.3. ¿Por qué el contador marcó más de 10 eventos con T_REBOTE = 0? ¿Qué cambió con 50 ms?

Porque el pulsador rebota y, sin filtro, cada rebote es un flanco que dispara la ISR: se contaron entre 905 y 963 eventos. Con 50 ms la ISR ignora los flancos que llegan muy seguidos al último aceptado [3], y el conteo bajó a 20; el segundo evento de cada toque viene del rebote al soltar, explicado en la Parte B.

6.4. ¿Por qué alerta y eventos son volatile y tUltimo no?

Las variables alerta y eventos se escriben en la ISR y se leen en el loop(); volatile obliga al compilador a leerlas siempre de memoria y no de una copia vieja en un registro. tUltimo solo lo usa la ISR, así que no hay otro código que pueda leer un valor desactualizado. Compartir datos entre la ISR y el programa principal sin este cuidado es un error típico de los sistemas embebidos [6].

6.5. ¿Por qué el Serial.print está en el loop() y no en la ISR?

Porque la ISR debe ser corta y Serial.print es lento y depende del búfer y del periférico UART, que trabajan con sus propias interrupciones. Si se moviera a la ISR, esta tardaría demasiado, podría quedarse esperando espacio en el búfer de la UART y el watchdog reiniciaría el ESP32 con un Guru Meditation Error; por eso se usa el patrón bandera [4].

6.6. Si se usara RISING en lugar de FALLING, ¿en qué momento se encendería la alarma?

Al soltar el pulsador, no al presionarlo. Con INPUT_PULLUP el pin pasa de 1 a 0 al presionar (FALLING) y de 0 a 1 al soltar (RISING), así que la alarma llegaría tarde: cuando la puerta ya se cerró.

7. Conclusiones

1. El sondeo cada 2 s detectó 0 y 1 aperturas de 10 en los intentos 1 y 2, mientras que la interrupción contó al menos un evento por toque (20 con T_REBOTE = 50) con el mismo delay(2000).
2. Sin antirrebote, 10 toques generaron entre 905 y 963 eventos; con T_REBOTE = 50 el conteo bajó a 20.
3. El antirrebote por tiempo no basta para toques más largos que T_REBOTE: el rebote al soltar se contó como un segundo evento en cada toque.

8. Recomendaciones

- Para toques más largos que T_REBOTE, aceptar el flanco solo si el pin sigue en 0 o usar un T_REBOTE mayor que la duración típica de un toque.
- Mantener el patrón bandera: la ISR solo registra y el loop() hace el trabajo lento.

9. Declaración de Uso de IA Generativa

Yo, Alejandro Padilla Espinoza, en nombre del grupo integrado por Melissa Contento, Jaime Landázuri, Doménica Rojas e Ivett Zaragocín, declaro de manera transparente el uso de herramientas de Inteligencia Artificial Generativa (IAG) en este trabajo.

9.1. Alcance del uso de IA generativa

La IAG se usó como apoyo, sin sustituir el juicio ni la autoría del grupo:

- Automatizar la simulación de Wokwi por línea de comandos (wokwi-cli) para registrar los conteos de las Tablas A y B a partir de los códigos de la guía, sin modificarlos salvo la constante T_REBOTE que pide la práctica.
- Extraer cuadros de los videos de la placa para contar los cambios de estado del LED de la Tabla C y obtener la foto del montaje.
- Estructurar, redactar y dar formato al informe.

9.2. Clasificación del uso según GAIDeT

- Nivel 1, asistencia lingüística: corrección y claridad de la redacción.
- Nivel 3, asistencia técnica: apoyo en la automatización de la simulación y en la construcción del documento.

9.3. Validación y responsabilidad

El grupo revisó los conteos contra los registros del Monitor Serie de cada simulación, comparó los cambios del LED con sus videos de la placa y asume la responsabilidad sobre el contenido.

9.4. Herramientas utilizadas

- Claude (Anthropic): automatización de la simulación, redacción y formato del informe. Partes afectadas: la redacción y el formato de todo el informe, y el contenido de Resultados, Preguntas de Control y Conclusiones.

Bibliografía / Referencias

[1] Espressif Systems. «ESP32 Series Datasheet,» visitado 6 de nov. de 2026. dirección: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[2] CodeMagic LTD. «Wokwi – documentación,» visitado 6 de nov. de 2026. dirección: https://docs.wokwi.com

[3] R. D. Yershov, V. P. Voytenko y V. A. Bychko, «Software-Based Contact Debouncing Algorithm with Programmable Auto-Repeat Profile Feature,» en *2019 IEEE International Scientific-Practical Conference Problems of Infocommunications, Science and Technology (PIC S&T)*, 2019. DOI: 10.1109/PICST47496.2019.9061500

[4] Espressif Systems. «Arduino-ESP32: GPIO (interrupciones externas),» visitado 6 de nov. de 2026. dirección: https://docs.espressif.com/projects/arduino-esp32/en/latest/api/gpio.html

[5] F. Reverter, «Toward Non-CPU Activity in Low-Power MCU-Based Measurement Systems,» *IEEE Transactions on Instrumentation and Measurement*, 2020. DOI: 10.1109/TIM.2019.2953374

[6] L. A. Freire Sánchez, *Electrónica y sistemas embebidos: una visión a nivel técnico y tecnológico*. Centro de Investigación y Desarrollo Ecuador, 2023, ISBN: 978-9942-636-22-5. DOI: 10.33996/cide.ecuador.ES2636225 dirección: https://repositorio.cidecuador.org/handle/123456789/2398

10. Anexos

La carpeta AP4 del repositorio (enlace en el esquema del circuito) contiene además los registros de texto del Monitor Serie de cada simulación por wokwi-cli y los escenarios de 10 toques usados.